



Token Bridge Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

jesjupyter

rotru

ihtisham-sudo

Taridoku

July 23, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
7	Findings	4
7.1	Medium Risk	4
7.1.1	Relayer cannot honor refundAddress	4
7.2	Low Risk	5
7.2.1	gasLimits has no destination-chain dimension	5
7.2.2	Batching TokenBridge.send silently diverts the relayer funding needed for leg 2	5
7.3	Informational	6
7.3.1	Spoke-to-spoke transfers can start with relayer == 0	6
7.3.2	Directly transferred shares are permanently stuck in TokenBridge	6
7.3.3	Existing-root migration omits the TokenBridge endorsement, disabling send for FullRestrictions shares	7

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

TokenBridge is a new Centrifuge Protocol contract that moves native share tokens across chains. It exposes a Glacis Airlift-compatible `send()` interface, takes custody of the caller's shares, maps the destination EVM chain ID to Centrifuge's internal chain ID, and forwards the transfer to `Spoke.crosschainTransferShares`, which burns and re-mints the shares on the destination chain.

For spoke → hub → spoke transfers, a configured relayer funds the second leg from the first leg's gas overpayment; when the hub is an endpoint the transfer is single-leg and the overpayment is refunded to the user. Per-pool gas limits are set via an authenticated `trustedCall`.

5 Audit Scope

The focus of the audit is `TokenBridge.sol`, but the other smaller changes introduced to files under `src/` in [PR #252](#) are also in scope.

Scope:

```
src/
├── admin/
│   ├── interfaces/
│   │   ├── IOpsGuardian.sol
│   │   └── IProtocolGuardian.sol
│   ├── OpsGuardian.sol
│   └── ProtocolGuardian.sol
└── bridge/
    ├── interfaces/
    │   └── ITokenBridge.sol
    └── TokenBridge.sol
```

6 Executive Summary

Over the course of 2 days, the Burra Security team conducted an audit on the [Token Bridge](#) smart contracts provided by [Centrifuge](#). In this period, a total of 6 issues were found.

Summary

Project Name	Token Bridge
Repository	protocol-internal
Commit	867bb92a0a19...
Audit Timeline	July 16th - July 17th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	2
Informational	3
Gas Optimizations	0
Total Issues	6

Summary of Findings

[M-1] Relay cannot honor <code>refundAddress</code>	Resolved
[L-1] <code>gasLimits</code> has no destination-chain dimension	Acknowledged
[L-2] Batching <code>TokenBridge.send</code> silently diverts the relayer funding needed for leg 2	Resolved
[I-1] Spoke-to-spoke transfers can start with <code>relayer == 0</code>	Resolved
[I-2] Directly transferred shares are permanently stuck in <code>TokenBridge</code>	Resolved
[I-3] Existing-root migration omits the <code>TokenBridge</code> endorsement, disabling <code>send</code> for <code>FullRestrictions</code> shares	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Relayer cannot honor `refundAddress`

Target: TokenBridge.sol, ITokenBridge.sol

Severity:

- Impact: Low
- Likelihood: High

Description: For spoke -> hub -> spoke transfers, TokenBridge._crosschainTransfer replaces the user-supplied `refundAddress` with the relayer. The original address is not forwarded, stored, or included in the `Send` event. An event-driven relayer therefore cannot distinguish `refundAddress` from `sender` when returning any surplus after funding leg 2. This contradicts the interface, which identifies `refundAddress` as the recipient of excess gas funds.

Recommended Mitigation: Include `refundAddress` in the `Send` event, or define `sender` as the refund recipient for two-leg transfers and document that behavior.

Centrifuge: Fixed in [centrifuge/protocol-internal#261](#)

BurraSec: Fix looks good. Refund address now in `Send` event.

7.2 Low Risk

7.2.1 gasLimits has no destination-chain dimension

Target: [TokenBridge.sol](#), [ITokenBridge.sol](#)

Severity:

- Impact: Low
- Likelihood: Medium

Description: `TokenBridge.gasLimits` is keyed only by `(poolId, scId)`. `send()` resolves the destination chain, but then reads the same `gasLimits[poolId][scId]` entry for every destination that share class supports.

```
mapping(PoolId => mapping(ShareClassId => GasLimits)) public gasLimits;
```

```
GasLimits memory limits = gasLimits[poolId][scId];
```

A pool that bridges the same share class to multiple spokes cannot set route-specific extra gas limits. Operators have to choose one `extraGasLimit` / `remoteExtraGasLimit` pair that works for all destinations, even if one destination needs more execution gas than another. This does not bypass the gateway's destination fee quote, but it does remove the ability to configure execution gas per route.

Recommended Mitigation: If different routes can require different execution gas, include destination `centrifugeId` in the gas limit key and in the trusted update payload. If one shared budget is intentional, document that constraint and monitor failed deliveries per route so the shared value can be adjusted before users are affected.

Centrifuge: For most cases, the hooks on the destination chains will likely be the same and so the execution gas also. Should they differ, the manager can account for the worst case.

BurraSec: Acknowledged

7.2.2 Batching `TokenBridge.send` silently diverts the relayer funding needed for leg 2

Target: `TokenBridge.sol`, `Gateway.sol`

Severity:

- Impact: Low
- Likelihood: Medium

Description: For spoke-> hub -> spoke transfers, `TokenBridge` routes the source-leg surplus to the relayer so it can repay the unpaid second leg. An integrator can call `TokenBridge.send{value: 0}` inside `Gateway.withBatch` and fund the outer batch instead. In batching mode, the per-message refund address is ignored and the surplus goes to the outer batch's refund address. The transfer still emits `Send`, but the relayer receives nothing. A receipt-verifying relayer will leave leg 2 underpaid, while an event-only relayer may fund it without compensation.

Recommended Mitigation: Make `TokenBridge.send` revert while the source `Gateway` is batching, and require the relayer to verify receipt before funding leg 2.

Centrifuge: Fixed in [centrifuge/protocol-internal#261](#)

BurraSec: Fix looks good. `TokenBridge` now reverts with `NotBatchable` if batching.

7.3 Informational

7.3.1 Spoke-to-spoke transfers can start with `relayer == 0`

Target: [TokenBridge.sol](#), [MessageDispatcher.sol](#), [HubHandler.sol](#), [Gateway.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: TokenBridge integrates a relayer for spoke → hub → spoke transfers: first-leg overpayment is held by the relayer and used to fund the second leg on the hub. NatSpec and the `relayer == 0` branch make this optional by design — when no relayer is set, overpayment is refunded to the user, and an underfunded second leg queues until Gateway repay. There is no permanent fund loss.

```
GasLimits memory limits = gasLimits[poolId][scId];
bool hubIsEndpoint = centrifugeId == poolId.centrifugeId() || localCentrifugeId == poolId.centrifugeId();
address refund = hubIsEndpoint || relayer == address(0) ? refundAddress : relayer;
spoke.crosschainTransferShares{value: msg.value}(
    centrifugeId, poolId, scId, receiver, amount, limits.extraGasLimit, limits.remoteExtraGasLimit, refund
);
```

The issue is that this weakens TokenBridge's stated expectation. The contract's purpose for spoke → hub → spoke is the relayer-funded second leg, yet `_crosschainTransfer` still accepts those transfers when `relayer == address(0)`. First-leg overpayment is then refunded on the source chain and cannot automatically fund the hub → destination leg, so completion depends on manual repay rather than the integrated relayer path.

Recommended Mitigation: Document clearly that spoke → spoke without a configured relayer relies on Gateway repay, and/or require `relayer != address(0)` (or an explicit user-funded both-legs path) before allowing spoke-to-hub-to-spoke transfers. Deployment should set `TokenBridge.relayer` before enabling those routes if the relayer UX is intended.

Centrifuge: Fixed in [centrifuge/protocol-internal#261](#)

BurraSec: Fix looks good, fallback documented.

7.3.2 Directly transferred shares are permanently stuck in TokenBridge

Target: [TokenBridge.sol](#), [ITokenBridge.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: Because the bridge is Root-endorsed, a normal transfer of restricted shares directly to the bridge may succeed. `FullRestrictions` treats a Root-endorsed recipient as an acceptable target. However, the bridge has no function that can bridge or return a preexisting balance. The protocol's `TokenRecoverer` can recover assets only by calling recovery functions on a target implementing `IRecoverable`.

Recommended Mitigation: Implement the protocol's standard `Recoverable` mechanism.

Centrifuge: Fixed in [centrifuge/protocol-internal#261](#)

BurraSec: Fix looks good, `TokenBridge` now implements `Recoverable`

7.3.3 Existing-root migration omits the TokenBridge endorsement, disabling `send` for `FullRestrictions` shares

Target: `FullDeployer.s.sol`, `MigrationSpell.sol`, `TokenBridge.sol`, `FullRestrictions.sol`

Severity:

- Impact: Medium
- Likelihood: Low

Description: `TokenBridge.send` first transfers the user's shares into `TokenBridge`. Under `FullRestrictions`, `TokenBridge` must therefore be a member, pool escrow, or Root-endorsed address. Fresh-root deployments endorse it, but existing-root deployments skip that step. `MigrationSpell._missingRootWards` endorses the other required contracts but omits `TokenBridge`, and its input does not contain the `TokenBridge` address. As a result, migrated deployments using `FullRestrictions` revert during the initial user-to-bridge transfer, before any cross-chain message is created.

Recommended Mitigation: Add `TokenBridge` to `GlobalParamsInput`, endorse it in the existing-root migration spell, and test `TokenBridge.send` with a `FullRestrictions` share token after migration.

Centrifuge: This won't be endorsed, rather added as a member.

BurraSec: Acknowledged. The protocol will add `TokenBridge` as a member rather than Root-endorsing it.